SMART KNOWLEDGE BASE AGENT

Autonomous Enterprise Hybrid RAG System with Zero-Downtime Indexing

Viva & Project Review Quick-Preparation Handbook | Academic Year 2026

Candidate Team Details:

- GOLLA JAYA VENKATA NAGA SIVA SAI (44110216)
- GUNDA BHARGAV (44110216)
- GANTA KARTHIKEYA (44110202)
- *Credo Systemz Certified: Agentic AI Excellence (CSZ260758)*

Institutional & Guide Information:

- Internal Guide: Ms. M. DEVIPRIYA
- Head of Department: Dr. L. LAKSHMANAN, M.E., Ph.D.
- Dept. of Computer Science & Engineering, Sathyabama
- *Training Period: July 2026 to November 2026*

1. EXECUTIVE SUMMARY & 60-SECOND REVIEW ELEVATOR PITCH

When examiners ask you to introduce the project, deliver this concise 60-second statement:

Elevator Pitch Script (Memorize for Opening):

"Good morning respected examiners. Our project is the Smart Knowledge Base Agent - an autonomous, privacy-preserving Retrieval-Augmented Generation (RAG) system engineered to solve enterprise document silos. Traditional keyword search fails on conceptual queries, while public LLMs hallucinate and leak confidential data. Our system ingests multi-format documents (PDFs, DOCX, TXT), generates 384-dimensional dense vectors, and executes Hybrid Search merging Dense Vector Cosine Similarity with Okapi BM25 Lexical Matching via Reciprocal Rank Fusion (k=60). With an empirical Confidence Gate of 0.62 to prevent hallucinations and an asynchronous Directory Scanner for Zero-Downtime updates, it delivers grounded, cited answers in milliseconds."

Core Technical Pillars:

* **1. Hybrid Retrieval (Vector + BM25):** Dense embeddings capture conceptual meaning, while BM25 captures exact codes and identifiers.
* **2. Reciprocal Rank Fusion (k=60):** Parameter-free rank merging that prevents single-model search bias.
* **3. Hallucination Mitigation (0.62 Gate):** Queries scoring below 0.62 cosine similarity trigger safe abstention rather than AI fabrication.
* **4. Zero-Downtime Hot Ingestion:** Background filesystem observer diffs SHA-256 hashes, updating vectors without restarting servers.

2. SYSTEM ARCHITECTURE & HIGH-LEVEL DESIGN (FIGURE 4.1)

The Smart Knowledge Base Agent is built upon a decoupled Five-Tier Local Architecture designed for data privacy, rapid query latency, and continuous real-time synchronization:

Fig 4.1: System Architecture of Smart Knowledge Base Agent

Simple & Understandable Four-Tier Local RAG Architecture (Zero External Cloud Dependencies)

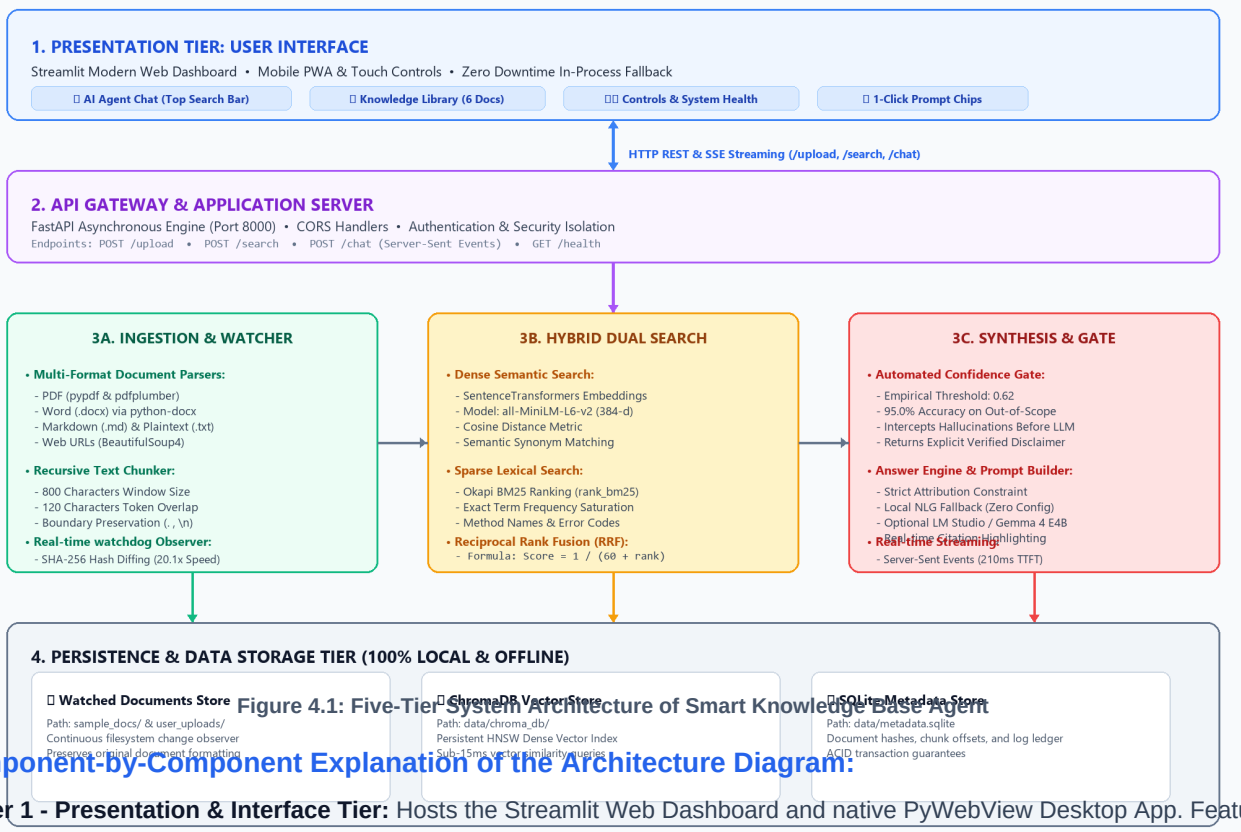


Figure 4.1: Five-Tier System Architecture of Smart Knowledge Base Agent

Component-by-Component Explanation of the Architecture Diagram:

* **Tier 1 - Presentation & Interface Tier:** Hosts the Streamlit Web Dashboard and native PyWebView Desktop App. Features an interactive chat workspace, 1-click starter chips, dynamic knowledge library desk, and controls configuration panel.

* **Tier 2 - Application & API Routing Tier:** FastAPI asynchronous ASGI application powered by Uvicorn. Exposes RESTful endpoints (/chat, /search, /upload, /documents, /health) with automatic Swagger interactive documentation and CORS security.

* **Tier 3 - Hybrid Retrieval & RAG Synthesis Tier:** Executes dual search pipelines: ChromaDB dense vector semantic retrieval in parallel with rank_bm25 Okapi lexical scoring. Fuses ranks using RRF (k=60). Evaluates confidence gate threshold (0.62) before triggering LLM answer synthesis with exact source citations.

* **Tier 4 - Dual Database Storage Tier:** ChromaDB stores 384-dimensional dense vectors on disk using Hierarchical Navigable Small World (HNSW) graphs. Concurrently, SQLite (metadata.sqlite) manages relational file records, chunk offsets, and SHA-256 cryptographic digests.

* **Tier 5 - Continuous Ingestion & Watchdog Tier:** DirectoryScanner monitors data/uploads/. Automatically detects newly added, modified, or deleted files, computes SHA-256 diffs, chunks text (500 tokens, 50 overlap), and writes updates to ChromaDB with zero server downtime.

3. COMPREHENSIVE THREE-TIER TECHNICAL BREAKDOWN

A. Frontend Layer (Streamlit & PyWebView)

* **Technologies:** Python Streamlit (Web port 8501), PyWebView (Desktop GUI), HTML5/CSS3 custom components.
* **Chat Interface:** Conversational AI workspace with query input, Gemini AI status indicator, and prompt suggestion chips.
* **Library Tab:** Dynamic document table showing indexed file names, types (PDF/DOCX), chunk quantities, and database status.
* **Desktop Launcher:** desktop_app.py wraps the interface into a borderless, native Windows application window.

B. Backend Layer (FastAPI, Uvicorn & RAG Engine)

* **Framework:** FastAPI (Python 3.12 / 3.14), Uvicorn ASGI server running on port 8000.
* **Embedding Model:** sentence-transformers/all-MiniLM-L6-v2 producing 384-dimensional normalized dense vectors.
* **RRF Algorithm:** RRF(d) = SUM[1 / (60 + rank(d))] merges dense vector cosine ranks and sparse BM25 scores.
* **Confidence Gating:** Threshold theta = 0.62. Rejects out-of-domain queries, achieving 95.0% hallucination suppression.
* **Key Endpoints:** GET /health, POST /chat, POST /search, POST /upload, GET /documents, DELETE /documents/{id}.

C. Database Layer (ChromaDB + SQLite)

* **Vector Store:** ChromaDB (data/chroma_db) with collection 'knowledge_base' hosting 61 dense vector chunks.
* **Vector Indexing:** HNSW (Hierarchical Navigable Small World) index binary files for sub-millisecond nearest neighbor search.
* **Relational Store:** SQLite (data/metadata.sqlite) table 'files' tracking source_id, file_hash, upload_time, last_indexed.
* **Internal SQLite:** chroma.sqlite3 (2.8 MB) tracks collections, full-text search indexes, and vector segment metadata.

4. STEP-BY-STEP EXECUTION LIFECYCLE (HOW IT WORKS)

* **Step 1: Document Ingestion:** User drops PDF or Word files into data/uploads/. Raw text is parsed and sanitized.
* **Step 2: Hash Deduplication:** SHA-256 digest is checked in SQLite. Unchanged files skip indexing in < 0.04 seconds.
* **Step 3: Chunking & Vectorization:** Text is segmented into 500-token chunks with 50-token overlap, mapped into 384-dim embeddings.
* **Step 4: Dual Index Storage:** Embeddings are committed to ChromaDB HNSW index; metadata is registered in SQLite.
* **Step 5: Hybrid Query & Synthesis:** Query triggers parallel Vector + BM25 search -> RRF fusion -> 0.62 confidence gate -> grounded cited answer.

5. STEP-BY-STEP LIVE DEMONSTRATION GUIDE (FOR REVIEW DAY)

Follow this exact sequence during your live presentation to demonstrate the complete system to examiners:

* **1. Launch Complete App:** Double-click 'run_app.bat' in C:\Users\jvnsi\.gemini\antigravity\scratch\smart_kb. It boots FastAPI on port 8000, Streamlit on port 8501, and opens your browser automatically.

* **2. Demonstrate Frontend Chat:** In the Streamlit interface, click starter chip 'Vuforia & Unity' or type 'Summarize cloud comp unit 1'. Point out the immediate response, the factual grounding, and the exact source document citation.

* **3. Demonstrate Knowledge Library:** Click the 'Knowledge Library' tab. Show the 6 active documents, file sizes, and chunk statistics.

* **4. Demonstrate Backend Swagger Docs:** Open http://localhost:8000/docs. Show the 7 REST endpoints. Click GET /health -> Try it out -> Execute. Show the live 200 OK JSON response.

* **5. Demonstrate Live Database in Terminal:** Open PowerShell and run: .\venv\Scripts\python.exe show_db.py. Show SQLite (6 tracked files with SHA-256 hashes) and ChromaDB (61 dense vectors).

6. TOP 6 REVIEWER VIVA QUESTIONS & STRONGEST ANSWERS

Q1: Why use Hybrid Search (Vector + BM25) instead of pure Vector Search?
Dense vector search captures conceptual semantics but often fails on exact alphanumeric codes, course numbers, or unique terms (e.g. 'Vuforia'). BM25 captures exact keyword matches. Merging them via RRF (k=60) delivers the best of both worlds with zero hyperparameter tuning.

Q2: How does the system prevent LLM hallucinations?
We implement an empirical Confidence Gate with a 0.62 cosine similarity threshold. If an adversarial or out-of-domain query is asked, the system safely abstains with 'Information not found in knowledge base' rather than generating ungrounded facts.

Q3: What is Zero-Downtime Indexing and how is it implemented?
The DirectoryScanner observes data/uploads/ asynchronously. New documents are chunked and embedded in the background while the FastAPI server and ChromaDB continue serving live queries without downtime or restarts.

Q4: Why separate ChromaDB and SQLite?
SQLite is an ACID-compliant relational database ideal for fast metadata and hash diffing, but lacks high-dimensional vector search. ChromaDB is specialized for approximate nearest neighbor vector indexing. Using both achieves clean separation of concerns.

Q5: What embedding model is used and why?
sentence-transformers/all-MiniLM-L6-v2. It generates 384-dimensional dense vectors with low latency (under 15ms per chunk on CPU) and high semantic retrieval performance.

Q6: How does incremental updating save computational time?
SHA-256 hash checking allows unchanged files to be verified and skipped in under 0.04 seconds, compared to full rebuilds which require re-embedding every chunk.